

PCS-302 : Data Structures Lab

Stack Operations Using the Minimum Number of Queues

Problem Statement

Design an algorithm and write a program to implement stack operations using the minimum number of queues.

Minimum Number of Queues Needed = 1

Even though a queue is FIFO and a stack is LIFO, a single queue can emulate a stack by rotating itself after every push, so the most recently pushed element is always at the front (ready to "pop" first).

Example

```
push(10)  --> Queue: [10]
push(20)  --> enqueue 20 -> [10,20] -> rotate once -> [20,10]
push(30)  --> enqueue 30 -> [20,10,30] -> rotate twice -> [30,20,10]
```

Stack (top to bottom): 30 20 10 (matches push order)

```
pop()      --> dequeue front = 30 (correct: last pushed, first out)
```

Algorithm (Using 1 Queue)

```
Push(k):
    enqueue(k)                // add new element at the back
    n = current size of queue
    repeat (n - 1) times:
        x = dequeue()
        enqueue(x)            // rotate old elements behind k
    // now k sits at the front

Pop():
    if queue is empty: print "Underflow", return
    return dequeue()          // front = most recently pushed = stack top

IsEmpty():
    return (queue is empty)
```

Each push does the rotation, so the front of the queue always behaves like the top of a stack.

C Program

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = 0, rear = -1, count = 0;

int isEmpty() {
    if (count == 0)
```

```

        return 1;
    else
        return 0;
}

void enqueue(int k) {
    if (count == MAX) {
        printf("Overflow. Cannot enqueue %d.\n", k);
        return;
    }
    rear = (rear + 1) % MAX;
    queue[rear] = k;
    count = count + 1;
}

int dequeue() {
    int val = queue[front];
    front = (front + 1) % MAX;
    count = count - 1;
    return val;
}

void push(int k) {
    int i, n;
    enqueue(k);
    n = count;                // total elements after adding k
    // rotate queue so that k moves to the front
    for (i = 1; i < n; i++) {
        int val = dequeue();
        enqueue(val);
    }
    printf("%d pushed to stack.\n", k);
}

void pop() {
    if (isEmpty() == 1) {
        printf("Stack Underflow.\n");
        return;
    }
    int val = dequeue();
    printf("%d popped from stack.\n", val);
}

void printStack() {
    int i, idx;
    if (isEmpty() == 1) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack (top to bottom): ");
    idx = front;
    for (i = 0; i < count; i++) {
        printf("%d ", queue[idx]);
        idx = (idx + 1) % MAX;
    }
}

```

```
        printf("\n");
    }

int main() {
    push(10);
    push(20);
    push(30);
    printStack();

    pop();
    printStack();

    return 0;
}
```